The container generation process is orchestrated by a *builder* image containing the complete information on how to containerize your application. Such information is provided as a sequence of *buildpacks*, each dedicated to a specific aspect of the application (such as the operating system, OpenJDK, and JVM configuration). The Spring Boot Plugin adopts the Paketo Buildpacks builder, an implementation of the Cloud Native Buildpacks specification that provides support for many types of applications, including Java and Spring Boot ones (https://paketo.io).

The Paketo builder component relies on a series of default buildpacks for the actual build operation. This structure is highly modular and customizable. You can add new buildpacks to the sequence (for example, to add a monitoring agent to the application), replace existing ones (for example, to replace the default Bellsoft Liberica OpenJDK with Microsoft OpenJDK), or even use a different builder image entirely.

NOTE The Cloud Native Buildpacks project manages a registry where you can discover and analyze buildpacks you can use to containerize your applications, including all the buildpacks from the Paketo implementation (https://registry.buildpacks.io).

The Buildpacks integration provided by the Spring Boot Plugin can be configured in the build.gradle file located in your Catalog Service project. Let's configure the image name and define which Java version to use through an environment variable.

Listing 6.5 Configuration for containerizing Catalog Service

```
bootBuildImage {                                    ❶
  imageName = "${project.name}"                     ❷
  environment = ["BP_JVM_VERSION" : "17.*"]         ❸
}
```

❶ The Spring Boot Plugin task to build an OCI image using Buildpacks

❷ The name of the OCI image to build. The name is the same one defined in the Gradle configuration for the project. We rely on the implicit "latest" tag when working locally rather than a version